



Université de Namur
Faculté d'Informatique

IEDC001 – Projet individuel

CondruLytics

Analyse, prédiction et visualisation des résultats
du Challenge Condrusien
de 2006 à 2025

Etudiant : Mestré Michaël
Professeur : Frénay Benoît
Année académique : 2025-2026

28 mai 2026

Table des matières

1	Introduction	1
1.1	Contexte	1
1.2	Objectifs	2
1.3	Etapes	2
2	Extraction des données et préparation	3
2.1	Architecture générale	3
2.2	Extraction des fichiers ZIP	3
2.3	Parsing des fichiers PDF	4
2.4	Consolidation	4
2.5	Nettoyage et harmonisation	4
2.6	Modèle de données	6
2.7	Service PostgreSQL avec Docker	8
2.8	Feature engineering	8
3	Prédiction	11
3.1	Prédiction à la demande	13
3.2	Arrivée de nouvelles données	13
4	Visualisation interactive	14
4.1	Vue d'ensemble	14
4.2	Courses	15
4.3	Coureurs	15
5	Conclusions	19
5.1	Limites identifiées	19
5.2	Perspectives d'amélioration	19
	Références	20

1 Introduction

Le projet *CondruLytics* a pour objectif de transformer des résultats historiques des courses du **Challenge Condrusien** en un jeu de données exploitable pour l'analyse, la visualisation et la modélisation. Les données de départ sont les différents classements au format PDF, publiés sur plusieurs années et selon des formats variables. Cette hétérogénéité rend nécessaire la mise en place d'un pipeline capable d'extraire les informations, de les harmoniser et de les stocker dans une base de données qui représentera la source stable pour la suite des analyses.

Le projet couvre les principales étapes d'un pipeline de données : extraction des fichiers, parsing des PDF, consolidation, nettoyage, chargement dans une base PostgreSQL, création de variables, apprentissage automatique pour prédire le temps des coureurs sur différentes courses et enfin visualisation interactive via le framework Python Streamlit ¹.

1.1 Contexte

Le **Challenge Condrusien** ² est un challenge de course à pied amateur sur routes et chemins vallonnés qui regroupe une vingtaine d'organisation chaque année dans le Condroz liégeois. Chaque organisation propose différentes distances : une petite sous les 7km, une longue entre 10 et 17km et parfois une version trail de plus de 20km. Les classements contiennent notamment le nom des coureurs, leur position générale, leur catégorie, leur temps, leur club, la distance de la course et la date de l'épreuve. Les fichiers ZIP de départ proviennent de la page officielle des archives du Challenge Condrusien ³. Les données d'archives couvrent la période 2006-2025 à l'exception de l'année 2020 en raison de la pandémie de COVID-19.

La difficulté principale provient du fait que les classements ne suivent pas un format unique. Les PDF présentent des structures différentes : colonnes absentes, ordre variable des champs, présence ou non du numéro de dossard, informations de course encodées dans le texte, ou encore changements de mise en page selon les années. Le projet répond à cette contrainte par l'utilisation de plusieurs parseurs spécialisés. Après cette étape d'extraction, les données sont nettoyées pour harmoniser les catégories, normaliser les temps et corriger les erreurs d'extraction. Le format des noms des coureurs sont standardisés et certaines erreurs de saisie sont corrigées. La base de données relationnelle créée permet ensuite de structurer les données pour des analyses, des expérimentations de modélisation et différentes visualisations interactives.

1. <https://streamlit.io>

2. <https://challengecondrusien.com>

3. <https://challengecondrusien.com/archives/>

1.2 Objectifs

Les objectifs du projet peuvent être regroupés selon les grands axes suivants :

1. **Construire un jeu de données fiable** à partir de fichiers PDF historiques.
2. **Harmoniser les informations** relatives aux coureurs, aux courses, aux catégories et aux performances.
3. **Mettre en place une base de données stable** permettant de consulter les résultats, les coureurs et les courses.
4. **Préparer des variables de modélisation** afin d'analyser l'évolution des performances et d'expérimenter des approches prédictives, notamment l'estimation du temps probable d'un coureur sur une course déjà présente dans l'historique mais qu'il n'a pas lui-même courue.
5. **Créer des visualisations interactives** pour permettre aux coureurs d'explorer les données, leur progression, la prédiction des résultats.

1.3 Etapes

Les objectifs précédents se traduisent en un pipeline de traitement structuré. Le projet part de fichiers d'archives, puis les transforme en une base exploitable pour l'analyse, la modélisation et la visualisation. La figure 1 résume les différentes étapes.

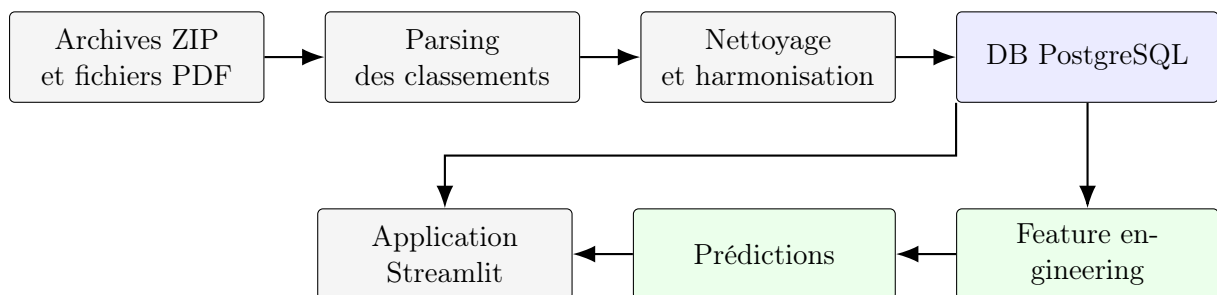


FIGURE 1 – Schéma des étapes du projet.

Les sections suivantes détaillent chacune de ces étapes.

2 Extraction des données et préparation

2.1 Architecture générale

Le projet est organisé sous forme de modules Python séparés par responsabilité. Cette décomposition facilite la maintenance et rend le pipeline reproductible.

Module	Role principal
<code>src/extraction</code>	Décompression des archives ZIP contenant les classements bruts.
<code>src/parsing</code>	Extraction des données depuis les fichiers PDF.
<code>src/cleaning</code>	Nettoyage, normalisation et correction des données extraites.
<code>src/loading</code>	Définition du schema relationnel et chargement vers PostgreSQL.
<code>src/utils</code>	Centralisation des fonctions utilitaires globales.
<code>src/feature_engineering</code>	Création de variables dérivées pour l'analyse avancée et la modélisation.

Un système de logging est mis en place pour suivre l'exécution du pipeline et identifier les éventuelles erreurs ou anomalies.

Le point d'entrée principal est le fichier `src/pipeline.py`. Celui-ci permet d'exécuter l'ensemble du pipeline ETL ou une étape spécifique via l'argument `-step`. Les étapes disponibles sont : `unzip`, `parse`, `build`, `clean` et `load`.

Les données intermédiaires sont stockées au format Parquet dans `data/processed/parquet`, ce qui permet de conserver une trace des résultats de chaque étape et de faciliter les reprises en cas d'erreur.

Les autres modules sont exécutés de manière séparée pour des tâches spécifiques, notamment la création de variables, ce qui rend le projet plus flexible et modulaire.

2.2 Extraction des fichiers ZIP

La première étape consiste à décompresser les archives ZIP situées dans `data/raw/zip`. Ces archives ont été récupérées depuis la page officielle des archives du Challenge Condru-sien⁴, qui centralise les classements historiques disponibles. Le module `extraction.unzip` parcourt récursivement les archives, extrait les fichiers et produit un rapport de synthèse. Ce rapport indique le nombre de fichiers extraits, le nombre de fichiers ignorés car déjà présents et le volume total traité.

4. <https://challengecondrusien.com/archives/>

2.3 Parsing des fichiers PDF

Le parsing est l'étape la plus sensible du projet. Les fichiers PDF sont convertis en tableaux de données grâce à la bibliothèque `pdfplumber`. Le projet utilise un patron de conception de type `Strategy` et un `ParserFactory`, choisit automatiquement le parseur adapté au fichier analyse.

Les stratégies implémentées permettent d'extraire toutes les données sur la plage de temps analysée et sont les suivantes :

- `Parser2006_2007`, adapté aux classements des années 2006 et 2007 ;
- `Parser2008_2016`, adapté aux formats intermédiaires sans URL dans le texte ;
- `Parser2017_2024`, adapté aux formats plus récents contenant des marqueurs web ;
- `Parser2025`, adapté aux classements produits par le chronometreur `Goal Timing`.

Chaque parseur extrait les champs principaux suivants : position, nom du coureur, club, sexe, catégorie, position dans la catégorie, temps, nom de la course, date et distance. Un identifiant de course, appelé `racekey`, est ensuite construit à partir du nom de la course, de la date et de la distance.

Le service de parsing traite les fichiers en parallèle avec `ThreadPoolExecutor`. Il génère également un rapport indiquant le statut de chaque fichier : succès, fichier ignoré, erreur ou différence entre le nombre de participants attendu et le nombre de lignes parsées.

Pour chaque fichier traité, un fichier Parquet est produit dans `data/processed/parquet` avec les données extraites. Ces fichiers intermédiaires sont ensuite utilisés pour les étapes suivantes du pipeline.

2.4 Consolidation

Après le parsing, les fichiers Parquet intermédiaires sont consolidés dans un dataset unique. Cette étape concatène les résultats disponibles dans `data/processed/parquet`, supprime les éventuels doublons sur base de `racekey` et du nom du coureur, puis écrit le fichier `data/processed/results.parquet`.

2.5 Nettoyage et harmonisation

Le nettoyage est effectué par le module `cleaning.clean`. Il vise à rendre les données homogènes malgré les variations historiques de format. Les principales opérations sont :

- harmonisation des catégories ;
- normalisation des temps au format `HH:MM:SS` puis conversion en secondes ;
- correction manuelle de certaines courses dont le nom ou la distance est mal extrait ;
- suppression de noms de coureurs invalides ou non exploitables ;

- correction ponctuelle de données coureurs connues ;
- normalisation des noms des coureurs afin de réduire les doublons ou erreurs orthographiques.

Une partie complexe du nettoyage concerne la correction du nom des coureurs. Dans les classements historiques, une même personne peut apparaître sous plusieurs formes : inversion du nom et du prénom, accentuation différente, caractères parasites, espaces multiples, fautes de frappe ou variations dues à l'extraction PDF. Se limiter au nom brut risquerait donc de considérer une même personne comme plusieurs individus distincts.

Le module `cleaning.clean_names` applique d'abord une normalisation des chaînes de caractères : passage en majuscules, suppression des caractères non alphabétiques (tirets, apostrophes) et réduction des espaces multiples. Cette opération crée une colonne `Nom_clean`, utilisée comme base de comparaison.

La correction ne repose cependant pas uniquement sur la similarité du nom. Le code construit un profil par coureur à partir de plusieurs informations :

- le nombre d'apparitions du nom dans les résultats ;
- le sexe majoritaire observé ;
- la vitesse moyenne du coureur ;
- l'ensemble des courses auxquelles le coureur est associé, pour les coureurs fréquents.

Cette logique permet d'éviter des corrections trop agressives. Deux noms proches ne sont pas automatiquement fusionnés : ils doivent aussi être compatibles avec le profil sportif observé. Par exemple, deux profils de sexe différent ne sont pas rapprochés. De même, deux coureurs fréquents présents dans une même course ne peuvent pas être considérés comme une seule et même personne, car une personne ne peut pas avoir deux résultats distincts dans la même épreuve.

La méthode fonctionne en deux passes.

1. **Première passe : coureurs fréquents entre eux.** Les coureurs apparaissant au moins trois fois sont comparés deux à deux. La similarité du nom est calculée avec un score de fuzzy matching à l'aide de la bibliothèque `RapidFuzz`. Ce score est combiné avec un score de proximité de vitesse. Le score final donne un poids dominant au nom, mais conserve une vérification par la performance. Si deux profils sont suffisamment proches, le nom le plus fréquent devient le nom corrigé. Cette première passe sur les coureurs fréquents permet de créer une base de noms fiables pour la suite du processus.
2. **Deuxième passe : coureurs rares vs coureurs fréquents.** Après la première correction, les noms apparaissant moins de trois fois sont comparés aux coureurs fréquents. Pour limiter les comparaisons inutiles, une clé de blocage est créée à partir des premières lettres des parties du nom. Le fuzzy matching est ensuite appliqué

uniquement à des candidats plausibles. Une correction est acceptée seulement si le score du nom et le score final dépassent des seuils élevés.

Cette approche est volontairement prudente. L'objectif n'est pas de corriger tous les cas possibles, mais uniquement les cas pour lesquels le niveau de confiance est élevé. Cela signifie que certains doublons peuvent subsister, notamment lorsque le nom est très déformé, lorsque le coureur apparaît peu de fois ou lorsque son profil de coureur est insuffisant. En contrepartie, le risque de fusionner deux personnes différentes est réduit. Ce compromis est important pour ce travail d'analyse : il est préférable de conserver quelques doublons résiduels plutôt que de fusionner accidentellement des individus distincts.

La fin de l'étape de nettoyage produit `data/cleaned/results.parquet`. Il est néanmoins nécessaire de noter que les corrections n'écrasent pas les données brutes, une nouvelle colonne `Nom_clean` est créée pour conserver le nom original et permettre de retracer les corrections appliquées.

2.6 Modèle de données

Les données nettoyées sont chargées dans une base de données PostgreSQL selon un schema en étoile simple compte tenu de la nature plate des données et des analyses prévues (figure 2). Cette organisation reprend les principes classiques de la modélisation dimensionnelle, dans laquelle une table de faits porte les mesures et les clés étrangères vers les dimensions descriptives.

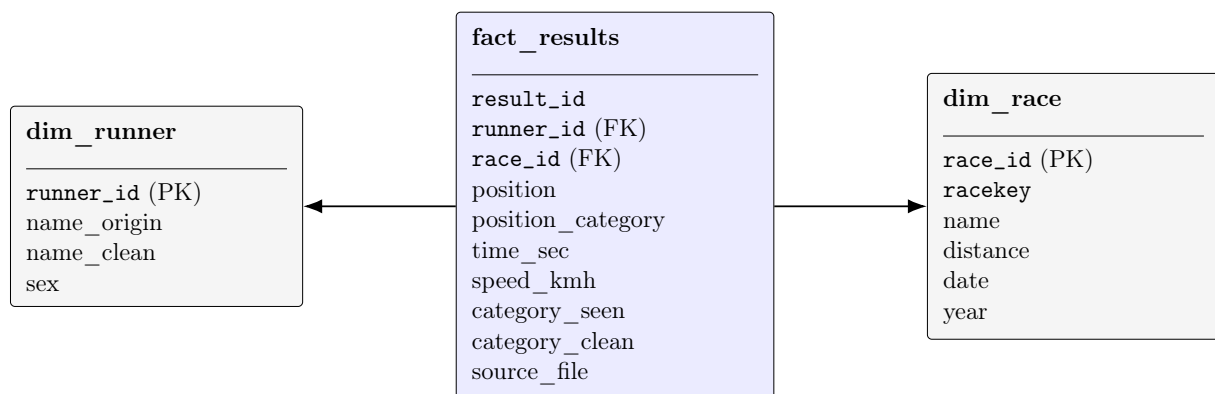


FIGURE 2 – Schéma en étoile utilisé pour structurer les résultats de courses.

Bien que la catégorie puisse être considérée comme étant un attribut du coureur, elle est stockée dans la table de faits car elle peut varier d'une course à l'autre pour un même coureur (par exemple, un coureur peut être dans la catégorie "Senior" puis basculer dans la catégorie "Vétérain" ensuite). De plus, cela permet de conserver une trace de la catégorie telle qu'elle a été observée dans les résultats, même si elle est parfois incohérente ou mal extraite.

Le schéma est ensuite étendu pour les besoins d'analyse et de prédiction. Pour rester dans une logique BI classique, cette extension prend la forme d'une seconde table

de faits, **fact_runner_race_analysis**. Elle a la même granularité que **fact_results** : une observation correspond à un coureur sur une course. Elle partage donc les mêmes dimensions **dim_runner** et **dim_race**, mais elle n'est pas modélisée comme une table fille de **fact_results**. La colonne **source_result_id** est conservée uniquement comme identifiant technique de traçabilité vers le résultat d'origine.

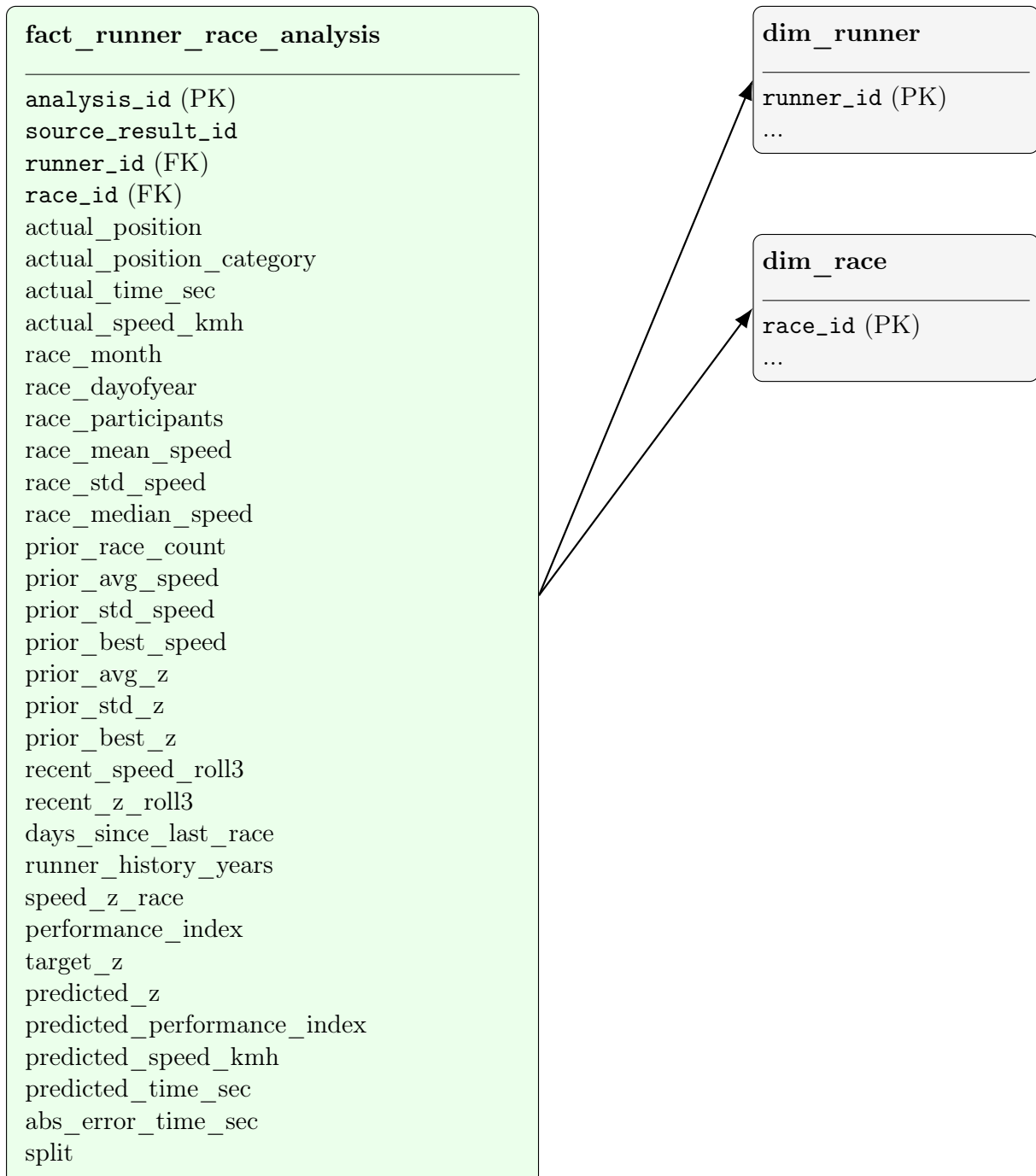


FIGURE 3 – Extension analytique sous forme de seconde table de faits partageant les mêmes dimensions.

Les prédictions à la demande, utilisées lorsqu'un coureur sélectionne une course qu'il n'a pas courue, ne sont pas stockées dans **fact_runner_race_analysis**. Elles ne corres-

pondent pas à un résultat observé dans l'historique. Elles sont donc calculées directement par l'application à partir du modèle entraîné.

Le chargement utilise des caches en mémoire pour un accès rapide et pour éviter de recréer plusieurs fois les mêmes coureurs ou les mêmes courses. Les résultats sont insérés par lots afin d'améliorer les performances d'écriture.

2.7 Service PostgreSQL avec Docker

Pour ce développement, la base de données PostgreSQL est fournie par un service Docker défini dans `docker/docker-compose.yml`. Ce fichier décrit un conteneur PostgreSQL nommé `condrulytics_postgres`, expose le port 5432 et crée une base `condrulytics` avec un utilisateur dédié. Un volume Docker, `postgres_data`, est associé au service afin de conserver les données entre deux redémarrages du conteneur. Cette approche permet d'utiliser rapidement une base de données locale, sans dépendre d'une installation PostgreSQL propre à une machine. Le code Python communique avec la base via SQLAlchemy et une configuration de connexion, ce qui signifie que la base Docker peut très facilement être remplacée par une base distante.

2.8 Feature engineering

Le module `feature_engineering.py` prépare un jeu de données destiné à deux objectifs : afficher l'évolution des performances d'un coureur dans l'interface finale et préparer une base de modélisation pour estimer un temps sur une course non encore courue. Cette étape est cruciale et ne doit pas utiliser des informations futures pour décrire un coureur au moment d'une course donnée, ce qui serait une fuite de données (*data leakage*) et rendrait les modèles non généralisables.

2.8.1 Principe général

L'idée est de ne pas seulement prédire une vitesse brute qui dépend fortement de la distance, du parcours, du dénivelé, de l'état du sol, de la météo et du niveau général des participants. Or plusieurs de ces variables ne sont pas disponibles dans les données. Pour tenter de contourner partiellement cette limite, un indice de performance normalisé est introduit.

Pour chaque résultat, la vitesse du coureur est comparée à la moyenne et à l'écart-type des vitesses observées sur la même course. Cette standardisation correspond au principe du z-score, utilisé pour exprimer une valeur en nombre d'écarts-types par rapport à une moyenne [1] :

$$z = \frac{vitesse_coureur - moyenne_course}{ecart_type_course}$$

La variable `speed_z_race` indique donc la position relative d'un coureur par rapport aux autres participants de la même épreuve. Une valeur positive signifie que le coureur a été plus rapide que la moyenne de cette course ; une valeur négative signifie qu'il a été plus lent. Ce score peut être vu comme un indice de performance relatif, ou normalisé, mais pas comme une mesure intrinsèque parfaite du niveau du coureur. Il reste dépendant du plateau de participants : une course très relevée ou très faible peut influencer le score.

Même sans disposer du dénivelé, de la météo ou du type de sol, tous les coureurs d'une même édition subissent globalement ces contraintes. On partira néanmoins de l'hypothèse que le niveau du plateau des participants reste similaire d'une course à l'autre. Le score normalisé permet donc de comparer plus proprement des performances réalisées sur des courses différentes qu'une simple vitesse brute.

2.8.2 Indice de performance

Pour l'interface finale, le z-score peut servir de base à un indice de performance lisible par les coureurs. Le code crée une variable `performance_index` selon l'échelle suivante :

$$indice = 100 + 10 \times z$$

Avec cette convention, un indice de 100 correspond à une performance moyenne dans la course. Un indice de 110 correspond à une performance située un écart-type au-dessus de la moyenne, et un indice de 90 à une performance située un écart-type en dessous. Cette transformation ne change pas l'information, mais elle produit une échelle plus lisible.

Cet indice sera utilisé dans l'application pour montrer l'évolution d'un coureur au fil du temps, indépendamment des distances exactes.

2.8.3 Variables historiques par coureur

Les variables descriptives du coureur sont calculées uniquement à partir des courses précédentes. Ce choix est important pour préparer une future prédiction : au moment de prédire le résultat d'un coureur sur une course donnée, on ne peut pas utiliser des résultats qui se produisent après cette course.

Le module calcule notamment :

- le nombre de courses déjà courues avant la course actuelle (`prior_race_count`) ;
- la vitesse moyenne, l'écart-type et la meilleure vitesse avant la course ;
- le z-score moyen, l'écart-type du z-score et le meilleur z-score avant la course ;
- des moyennes glissantes sur les trois dernières courses, en vitesse et en z-score ;
- le nombre de jours depuis la dernière course ;
- l'ancienneté du coureur dans l'historique disponible.

Ces variables décrivent à la fois le niveau global du coureur, sa régularité, sa forme récente et son expérience. Le décalage d’une ligne avant le calcul des moyennes permet d’éviter la fuite de données (*data leakage*) : la performance courante n’est jamais utilisée pour expliquer ou prédire cette même performance.

2.8.4 Variables liées à la course

Le mois et le jour de l’année de la course sont ajoutés. Ces variables peuvent capter partiellement des effets saisonniers, par exemple le fait que certaines courses se déroulent toujours à une période spécifique de l’année.

2.8.5 Sortie des features

Cette étape génère d’abord un fichier `data/features/features_ml.parquet`. Ce fichier reste pratique pour des vérifications locales et les essais dans un notebook jupyter. Pour le reste du projet, les features sont aussi enregistrées dans PostgreSQL, dans la table `fact_runner_race_analysis`. Cette table contient une ligne par couple coureur-course et partage les dimensions `dim_runner` et `dim_race` avec la table de faits principale. La colonne `source_result_id` permet uniquement de conserver une trace du résultat d’origine.

Le module est lancé manuellement afin de garder une démarche simple :

```
python -m src.feature_engineering.feature_engineering --save-db
```

3 Prédiction

L’objectif final de la plateforme est double : permettre à un coureur de consulter ses résultats et son évolution (estimer si pour une course donnée, il sûr ou sous performe par rapport à ses résultats passés), mais aussi estimer son temps probable sur une course qu’il n’a pas courue dans l’historique disponible. Cette prédiction doit être comprise comme une simulation rétrospective : la course cible existe déjà dans la base et a donc été courue par d’autres participants. Le modèle utilise l’historique individuel du coureur et les caractéristiques connues de la course pour produire une estimation.

Ce choix est lié aux limites des données disponibles. Les fichiers ne contiennent pas le dénivelé, l’état du sol, la météo ou d’autres caractéristiques fines du parcours. Le projet contourne partiellement cette limite en raisonnant sur une performance relative : le coureur est comparé au niveau observé sur une même course. En revanche, le modèle actuel ne permet pas de prédire une course future avant qu’elle ait lieu, car la moyenne et l’écart-type des vitesses de cette édition ne sont alors pas connus. Pour une course qui se répète sur plusieurs années, l’approche peut toutefois aider un coureur à se situer par rapport à ses performances passées ou par rapport aux coureurs ayant déjà participé à une édition disponible dans l’historique.

L’objectif est donc de prédire le z-score attendu du coureur sur une course, puis à convertir ce z-score en vitesse et en temps.

La conversion peut se faire de la manière suivante :

$$vitesse_predite = moyenne_course + z_predict \times ecart_type_course$$

$$temps_predict = \frac{distance}{vitesse_predite} \times 3600$$

Le projet contient un module de régression (`src/modeling/regression.py`) qui entraîne une régression linéaire pour prédire `target_z`.

Les variables utilisées sont les suivantes : distance de la course, mois et jour de l’année, nombre de courses précédentes, vitesse moyenne passée, régularité, meilleur niveau observé, performance récente sur trois courses, temps depuis la dernière course et ancienneté du coureur dans l’historique.

Le découpage train/test est temporel : les années anciennes servent à l’entraînement et l’année la plus récente (2025) sert au test.

Ce modèle a été préalablement expérimenté dans un notebook Jupyter. Cette démarche permet de tester différentes combinaisons de variables, d’identifier les variables les plus influentes et de vérifier que les résultats sont cohérents avant de les intégrer dans un script Python.

Ce modèle a été évalué selon deux variables de sortie : le z-score prédit et le temps reconstruit. Les métriques principales sont l’erreur absolue moyenne, la RMSE et le coefficient R^2 . Le notebook calcule ces valeurs séparément sur le jeu d’entraînement et sur le jeu de test afin d’identifier un éventuel surapprentissage.

Jeu	n	MAE z	RMSE z	R^2 z	MAE temps (s)	RMSE temps (s)
Entraînement	74589	0,331	0,481	0,771	217	474
Test	5627	0,335	0,473	0,778	200	314

TABLE 2 – Résultats de la régression linéaire sur les données disponibles.

Les résultats ne montrent pas de surapprentissage évident : le score de test reste proche du score d’entraînement. L’erreur moyenne sur le temps prédit est d’environ trois à quatre minutes. Cette valeur doit toutefois être interprétée avec prudence, car elle dépend fortement de la distance des courses. Une erreur de 200 secondes sur une course de 30 minutes est plus significative que la même erreur sur une course de 2 heures. L’utilisation du z-score comme variable cible permet de contourner partiellement cette limite, car elle mesure une performance relative plutôt qu’une valeur absolue. De plus, cette erreur de temps doit être mise en perspective avec la variabilité des performances d’un coureur, qui peut être influencée par de nombreux facteurs non capturés dans les données (forme du jour, conditions météo, état du parcours, etc.).

Variable	Coefficient standardisé
<code>recent_z_roll3</code>	0,512
<code>prior_avg_speed</code>	0,289
<code>distance</code>	-0,139
<code>prior_avg_z</code>	0,106
<code>race_month</code>	-0,055
<code>recent_speed_roll3</code>	0,054

TABLE 3 – Variables les plus influentes pour le modèle de régression linéaire.

Les variables les plus influentes sont la performance récente du coureur (moyenne des trois derniers z-scores), la vitesse moyenne passée, la distance de la course et la performance moyenne passée. Ces résultats sont cohérents avec l’intuition : un coureur qui a été rapide récemment et qui a une bonne moyenne de vitesse dans le passé est plus susceptible d’avoir un bon z-score sur une nouvelle course. La distance de la course a un impact négatif, ce qui peut s’expliquer par le fait que les performances relatives ont tendance à être plus faibles sur les longues distances, où la variabilité est plus grande.

Les prédictions historiques sont stockées dans PostgreSQL, directement dans la table `fact_runner_race_analysis`. Les colonnes `predicted_z`, `predicted_time_sec` et `abs_error_time_s` complètent alors les features et les mesures réelles du même résultat. Cela permet de com-

parer les temps réels et les temps estimés dans l'interface sans créer une relation directe entre les deux tables de faits. Le modèle lui-même est sauvegardé localement.

Tout comme le feature engineering, la génération des prédictions historiques est lancée manuellement afin de garder une démarche simple :

```
python -m src.modeling.regression --save-db
```

3.1 Prédiction à la demande

Pour estimer le temps qu'un coureur aurait pu réaliser sur une course qu'il n'a pas courue, deux approches sont possibles. La première consisterait à précalculer toutes les combinaisons coureur-course. Cette solution peut devenir volumineuse et produire beaucoup de prédictions qui ne seront jamais consultées. La seconde consiste à prédire à la demande, au moment où l'utilisateur sélectionne un coureur et une course dans l'interface.

C'est cette seconde approche qui a été retenue pour sa simplicité de mise en oeuvre et son besoin d'espace limité. Le module `src/modeling/serving.py` charge le modèle de régression sauvegardé, construit les variables nécessaires pour le couple coureur-course sélectionné, puis renvoie une prédiction du z-score, de la vitesse et du temps à la condition que le coureur ait déjà couru au moins trois courses avant la course cible.

3.2 Arrivée de nouvelles données

Si de nouvelles données des années futures devaient être ajoutées, la stratégie retenue est un recalcul complet. Lorsque de nouveaux classements sont ajoutés, le pipeline recharge les résultats, crée la table de features, génère le modèle et les prédictions historiques sont recalculés. Cette approche est moins optimisée qu'un traitement incrémental, mais elle est plus simple, contrôlée et suffisante pour le volume de données que représente le Challenge Condrusien.

4 Visualisation interactive

Le projet inclut une application Streamlit nommée *CondruLytics*. Elle interroge la base PostgreSQL et propose plusieurs vues interactives pour explorer les données et les prédictions. L'application est organisée en trois sections principales :

- une vue d'ensemble ;
- une vue par course ;
- une vue par coureur.

4.1 Vue d'ensemble

La vue d'ensemble sert de point d'entrée dans l'application. Elle affiche d'abord quelques indicateurs globaux, comme le nombre de coureurs, de courses, de résultats et la période couverte par les données. Elle permet ensuite d'observer la dynamique du challenge au fil des années grâce à un graphique en barres représentant le nombre de résultats par année.

Lorsqu'une année est sélectionnée, la liste des courses correspondantes est affichée à droite avec leur distance et leur nombre de participants. Cette sélection est également répercutée dans le graphique des courses et distances : les points associés à l'année active sont mis en évidence avec la même couleur que la barre sélectionnée. Le nom des courses et les points du second graphique sont également cliquables pour permettre de passer d'une vue globale vers une vue plus détaillée.

Vue d'ensemble

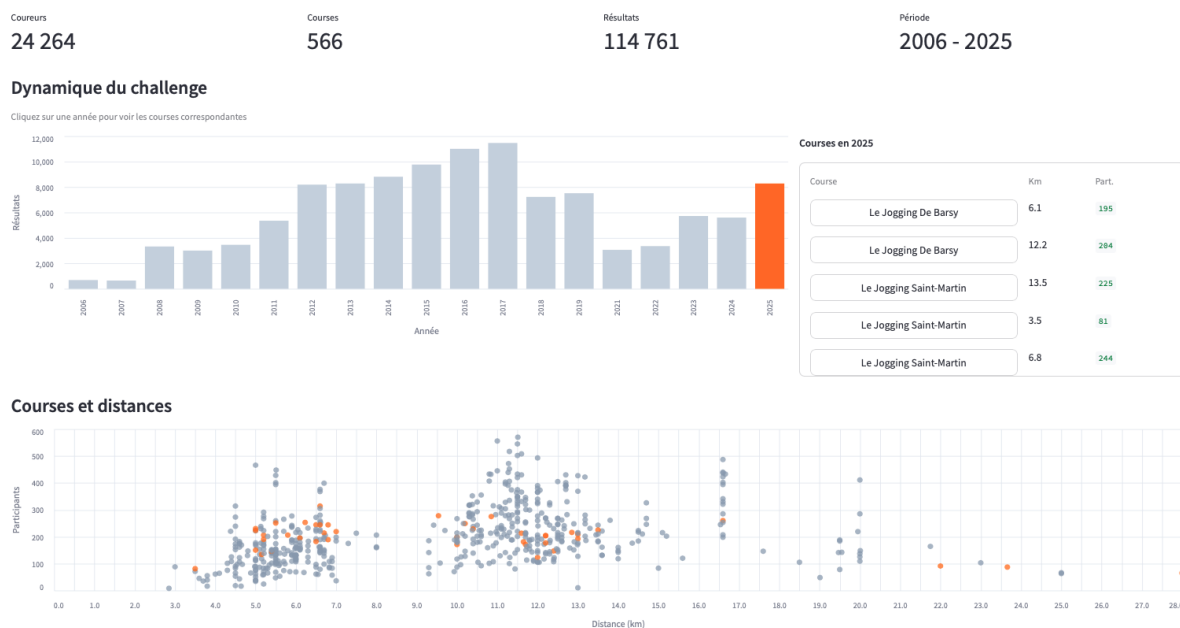


FIGURE 4 – Vue d'ensemble de l'application CondruLytics.

4.2 Courses

La vue par course permet de consulter les résultats détaillés d'une épreuve précise. L'utilisateur choisit d'abord une course parmi les années disponibles. Des indicateurs présentent ensuite le nombre de participants, la distance, le temps moyen et la vitesse moyenne observée sur cette course.

Le premier onglet contient le classement qui reprend les caractéristiques principales. Lorsque des prédictions historiques sont disponibles, le tableau ajoute également le temps attendu et l'écart entre le temps réalisé et le temps attendu. Une valeur négative signifie que le coureur a été plus rapide que ce que le modèle prévoyait. Un filtre par catégorie permet de limiter l'affichage à un groupe précis, ce qui facilite la lecture du classement pour un coureur qui souhaite se comparer à sa catégorie.

L'onglet d'analyse complète le classement avec une lecture globale de la course. Il présente la répartition des participants par catégorie, puis un tableau de synthèse de différentes caractéristiques de la course selon les catégories.

Courses

Course

Course Nature Du Grand Bois D'Anthisnes - 2025 (5.00 km)

Participants

Distance

Temps moyen

Vitesse moyenne

150

5.00 km

31:58

9.80 km/h

Classement

Analyse

Cliquez sur le nom d'un coureur pour ouvrir sa fiche.

Catégorie

Toutes

L'écart est calculé comme temps réalisé moins temps attendu. Une valeur négative signifie que le coureur a été plus rapide que prévu.

Gén.	Cat.	Coureur	Catégorie	Temps	Attendu	Écart	Allure	Vit.	Indice
1	1	GREGOIRE VALENTIN	SH	19:03	18:50	+0:13	3:49/km	15.75	128.5
2	2	BILLA GARCIA SAMUEL	SH	19:47	20:36	-0:49	3:57/km	15.16	125.7
3	3	GUILLLOT EVAN	SH	19:55			3:59/km	15.06	125.2
4	1	BOSMAN NATHAN	ESH	20:11	22:52	-2:41	4:02/km	14.86	124.2
5	1	FORET TOM	ESH	20:37	21:49	-1:12	4:07/km	14.55	122.8
6	1	MARTIN YANNICK	V1	21:14	22:57	-1:43	4:15/km	14.13	120.7
7	1	ROUHART DANIEL	V2	21:53	24:50	-2:57	4:23/km	13.71	118.7
8	4	WASSON CORENTIN	SH	22:04	26:36	-4:32	4:25/km	13.60	118.2

FIGURE 5 – Classement détaillé d'une course dans l'application CondruLytics.

4.3 Coureurs

La vue coureur permet d'analyser l'historique individuel d'un participant. Après sélection d'un coureur, l'application affiche quelques indicateurs de synthèse : nombre de courses réalisées, distance totale, allure moyenne, indice moyen et meilleure performance relative.

L'onglet *Profil* présente l'évolution de l'indice de performance au fil du temps. Lorsque

Courses

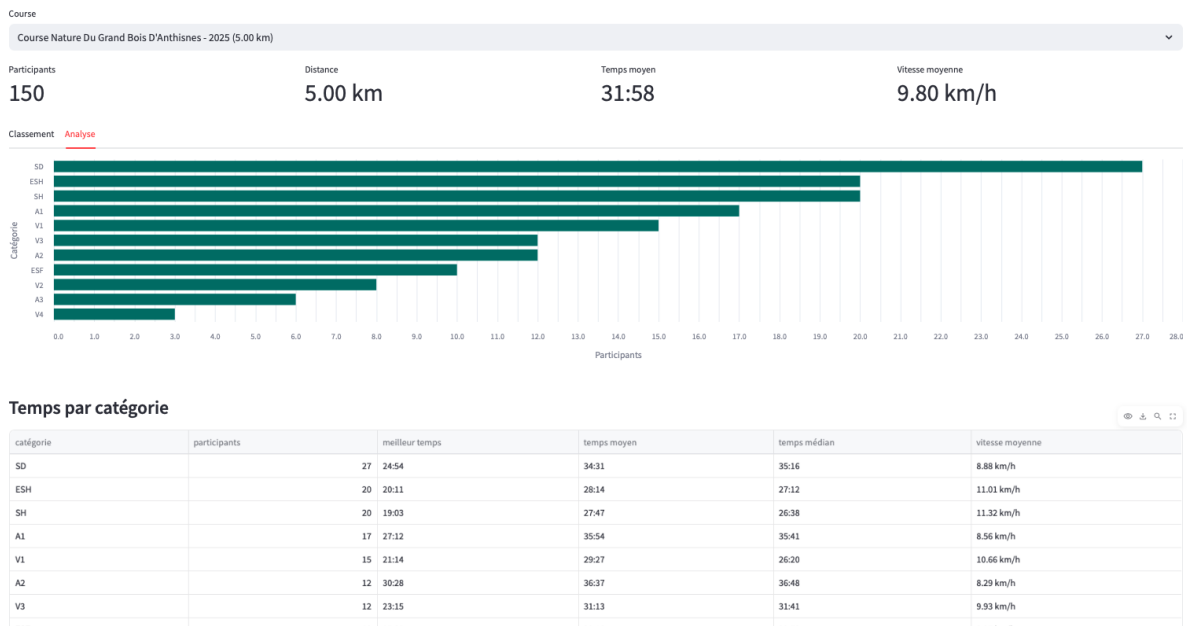


FIGURE 6 – Analyse par catégorie d’une course dans l’application Condrulytics.

les prédictions historiques sont disponibles, l’indice attendu est superposé à l’indice réellement observé, ce qui permet de voir rapidement les courses où le coureur a fait mieux ou moins bien que prévu. Un second graphique donne les performances selon la distance des courses, afin d’analyser la distance de prédilection du coureur.

L’onglet *Simulation* permet d’estimer rétrospectivement le temps qu’un coureur aurait pu réaliser sur une course déjà présente dans l’historique mais qu’il n’a pas courue. L’application affiche alors un temps estimé, une allure, une vitesse et un indice de performance estimé.

Enfin, l’onglet *Détail* reprend l’ensemble des résultats du coureur sous forme de tableau. Le nom de la course est cliquable et permet de revenir à la page détaillée de cette course.

Coueurs



FIGURE 7 – Profil d'un coureur dans l'application CondruLytics.

Coueurs

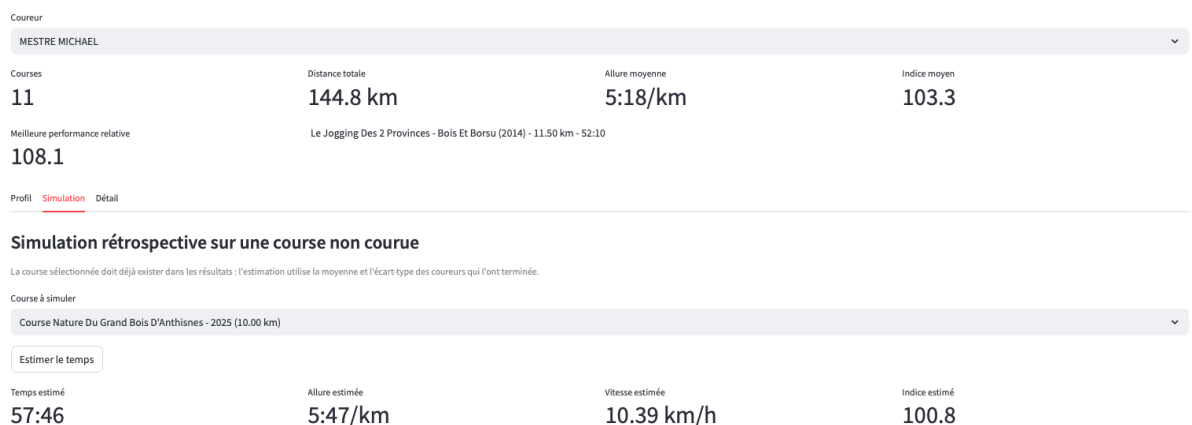


FIGURE 8 – Simulation rétrospective d'une course non courue.

Coueurs

Coureur

MESTRE MICHAEL

Courses

11

Distance totale

144.8 km

Allure moyenne

5:18/km

Indice moyen

103.3

Meilleure performance relative

108.1

Le Jogging Des 2 Provinces - Bois Et Borsu (2014) - 11.50 km - 52:10

Profil Simulation **Détail**

Cliquez sur le nom d'une course pour ouvrir sa page.

Date	Course	Distance	Pos.	Catégorie	Temps	Allure	Vit.	Indice	Attendu
2025-08-15	Le Jogging Du Plateau De Frisee	7.00 km	85	V1	44:51	6:24/km	9.36	101.7	43:17
2025-06-08	La Clavieroise	6.23 km	181	V1	36:18	5:50/km	10.30	101.3	34:41
2023-08-05	Autour De Villers-Aux-Tours	6.00 km	73	V1	38:47	6:28/km	9.28	96.4	33:42
2016-09-24	Le Jogging De L'Ecole - Fraiture-En-Condroz	19.50 km	23	SH	1:31:17	4:41/km	12.82	106.5	1:39:45
2015-02-28	Jogging Saint Martin - Nandrin	20.01 km	74	SH	1:51:14	5:34/km	10.79	100.1	1:47:12
2014-09-27	Lj Binameye - Fraiture	19.95 km	90	SH	1:46:05	5:19/km	11.28	101.8	1:43:00
2014-09-06	La Foulee Anthinoise - Anthisnes	12.30 km	52	SH	1:01:18	4:59/km	12.04	107.6	1:01:49
2014-06-13	La Condruisienne - Solieres	11.50 km	55	SH	54:05	4:42/km	12.76	107.8	55:34
2014-05-03	Le Jogging Des 2 Provinces - Bois Et Borsu	11.50 km	76	SH	52:10	4:32/km	13.23	108.1	54:48
2014-03-16	La Neupreenne - Neupre	11.30 km	128	SH	53:03	4:42/km	12.78	107.0	
2013-09-28	Le Binameye - Fraiture En Condroz	19.47 km	71	SH	1:38:43	5:04/km	11.83	98.4	

FIGURE 9 – Détail des résultats d'un coureur.

5 Conclusions

Ce projet part de classements historiques du Challenge Condrusien au format PDF hétérogène pour les transformer en une base de données exploitable et en une application interactive. Cela couvre l'extraction, le nettoyage, la sauvegarde dans une base de données relationnelle, la création de variables, la modélisation prédictive et la visualisation. Son principal objectif est de rendre des données sportives dispersées consultables par course, par coureur et par année, tout en ajoutant des indicateurs de performance et des simulations rétrospectives. Les prédictions doivent cependant être vues comme une aide à l'analyse, à des suivi de tendance et non comme une mesure définitive de la performance.

5.1 Limites identifiées

- Les PDF restent hétérogènes, ce qui peut entraîner des erreurs de parsing ou des corrections manuelles.
- L'identification des coureurs repose surtout sur le nom nettoyé, avec un risque d'erreur en cas d'homonymes ou de variations d'orthographe.
- La simulation est rétrospective : la course cible doit déjà exister dans l'historique.

5.2 Perspectives d'amélioration

- Ajouter des tests automatisés sur le parsing, le nettoyage et les transformations.
- Améliorer l'identification des coureurs.
- Prévoir une stratégie pour estimer des courses futures à partir d'éditions précédentes ou de courses comparables.

Références

Références

- [1] OpenStax, “The Standard Normal Distribution”, *Introductory Statistics*. <https://openstax.org/books/statistics/pages/6-1-the-standard-normal-distribution>